



An Efficient White-box LLM Watermarking for IP Protection on Online Market Platforms

Shuguang Yuan

Institute of Information Engineering,
Chinese Academy of Science
State Key Laboratory of Cyberspace
Security Defense
Beijing, China
yuanshuguang@iie.ac.cn

Xingyu Su*

Institute of Information Engineering,
Chinese Academy of Science
State Key Laboratory of Cyberspace
Security Defense
Beijing, China
suxingyu@iie.ac.cn

Peizhuo Lv

Nanyang Technological University
College of Computing and Data
Science
Singapore, Singapore
peizhuo.lyu@ntu.edu.sg

Weiji Xue[†]

Institute of Information Engineering,
Chinese Academy of Science
State Key Laboratory of Cyberspace
Security Defense
Beijing, China
xueweiji@iie.ac.cn

Jing Yu[‡]

Institute of Information Engineering,
Chinese Academy of Science
State Key Laboratory of Cyberspace
Security Defense
Beijing, China
yujing@iie.ac.cn

Xiaojie Zhu[§]

King Abdullah University of Science
and Technology
Jeddah, Saudi Arabia
xiaojie.zhu@kaust.edu.sa

Chi Chen[¶]

Institute of Information Engineering,
Chinese Academy of Science
State Key Laboratory of Cyberspace
Security Defense
Beijing, China
chenchi@iie.ac.cn

Abstract

Online market platforms serve as a central hub for sharing and deploying AI models among researchers, developers, and companies. In this context, watermarking techniques are essential to protect intellectual property (IP), preventing unauthorized use and duplication of large language models (LLMs). Two key challenges arise: (i) These platforms host diverse LLMs, yet current watermarking techniques are only tailored to specific models, such as fine-tuned or quantized LLMs. (ii) Efficient watermarking is critical. However, traditional methods require substantial data and costly hardware, which limits their feasibility. In this paper, we propose an efficient white-box LLM watermarking technique called ELLMark. This method treats LLMs as multi-layered matrices while embedding watermarks only relies on modifying the model's weights. To preserve LLMs' performance, it filters weights by correlations with

the activation magnitudes and downstream tasks, then modifies weights as minimal as possible via histogram modulation. Notably, all phases are training-free with low hardware resources, making it efficient for online platforms. We conduct extensive experiments to evaluate the effectiveness of ELLMark on LLaMA-3, OPT, and Phi-3 LLMs. The results demonstrate that it achieves 100% success in watermark detection while preserving model performance. Moreover, the preprocessing, encoding, and decoding processes remain efficient, taking less than 7 minutes, 12 minutes, and 18 seconds, respectively, for models with 80B parameters. Lastly, it exhibits robustness against parameter overwriting, re-watermarking, forging, fine-tuning, and pruning attacks.

CCS Concepts

• **Information systems** → Web services; • **Computing methodologies** → *Artificial intelligence*; • **Security and privacy**;

Keywords

Watermarking; LLM; Robust; IP Protection; Online Market Platform

ACM Reference Format:

Shuguang Yuan, Xingyu Su, Peizhuo Lv, Weiji Xue, Jing Yu, Xiaojie Zhu, and Chi Chen. 2025. An Efficient White-box LLM Watermarking for IP Protection on Online Market Platforms. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '25)*, August 3–7, 2025, Toronto, ON, Canada. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3711896.3736841>

* Also with School of Cyber Security, University of Chinese Academy of Science.

[†] Also with School of Cyber Security, University of Chinese Academy of Science.

[‡] Also with School of Cyber Security, University of Chinese Academy of Science.

[§] Corresponding author: Xiaojie Zhu

[¶] First Corresponding author: Chi Chen

[‡] Also with School of Cyber Security, University of Chinese Academy of Science.



This work is licensed under a Creative Commons Attribution 4.0 International License. *KDD '25, Toronto, ON, Canada*

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1454-2/2025/08

<https://doi.org/10.1145/3711896.3736841>

1 Introduction

Online market platforms, such as ModelZoo [2], ONNX [3], and Hugging Face [1] are marketplaces where businesses and individuals can access, share, and deploy AI solutions and services. Large Language Models (LLMs) are a category of AI models trained on immense amounts of data, making them capable of understanding and generating natural language to perform a wide range of tasks [4], including chatbots [6], enhanced recommendations [46] and content creation [12, 43]. The LLMs hold significant intellectual property (IP) value due to the substantial investments in hardware, energy consumption, vast datasets, and human resources [31].

Watermarking techniques are essential to prevent duplication, unauthorized resale, and commercial exploitation of the LLMs. Implementing watermarking protection serves multiple purposes: it safeguards user interests, demonstrates platform commitment to anti-piracy measures, and fosters a sustainable AI ecosystem. Thus, there is a pressing need to develop a watermarking solution. In online platforms, it is common to encounter situations where stolen models are commercialized or released, providing potential access to their parameters and structure, known as a white-box scenario. For this goal, it faces two key challenges:

- **Model Adaptability.** Firstly, in the online marketplace platform, there exist various customized LLMs. LLMs differ in architectures (e.g., decoder-only, encoder-decoder), precision (e.g., FP32, FP16, INT8), and parameter sizes (e.g., 8B, 14B, 80B), especially between full-precision and quantized models. However, traditional methods for deep neural networks (DNNs) [11, 16, 19, 28, 35, 41, 42, 50], diffusion models (DMs) [49], and pre-trained language models (PLMs) [23, 27, 34] are hard to extend their capability to handle diverse tasks, vast datasets, different architectures and billion parameters. Thus, they can not be applied to diverse LLMs. Besides, existing LLM approaches [22, 24, 32, 47] have adaptability limitations. For instance, [24, 32] are only suitable for LoRA fine-tuning models, and [47] targets quantized models on resource-constrained devices. Secondly, users are highly sensitive to the usability of models. Watermarking must not significantly degrade the model's performance. For diverse LLMs, watermarking requires a generic method to evaluate and implement embedding.
- **Watermark Efficiency.** Embedding or detecting watermarks must be performed efficiently. In this scenario, the watermarking system needs to process a large number of LLMs. This efficiency is particularly crucial in that the watermarking process should not become a bottleneck for sharing or deploying LLMs. However, traditional watermarking methods [11, 16, 19, 22, 24, 28, 35, 41, 42, 50] generally embed specific watermarks via training. With billions or even trillions of parameters of LLMs, the training process requires the preparation of enormous datasets and high hardware costs, which can not ensure an acceptable time. These high requirements limit the application of watermarking.

We present an efficient white-box LLM watermarking technique on the online market platform called **ELLMark**. ELLMark treats LLMs as multi-layered matrices while embedding watermarks only relies on the model's weights. This strategy ensures adaptability to structural variations in LLMs. By analyzing the weight activation during LLM inference, ELLMark assesses the importance of weights and embeds the watermark into suitable weights without

training. Thus, this approach guarantees efficient watermarking while preserving the LLM's performance.

The watermark process contains three phases: watermark preprocessing, encoding, and decoding. In the preprocessing phase, we filter weights based on activation magnitude, highlighting significant contributions of weights [25] and correlations with downstream tasks, ensuring minimal impact on main task performance. Additionally, a random filtering mechanism can result in a larger portion of salient weight parameter preservation. After filtering, the remaining weights are candidates for subsequent embedding. In the encoding phase, we introduce a flexible locating technique that determines watermark by leftmost bits and embeds information into the rightmost bits of weight. It is adaptable for LLMs with different precision and parameter sizes. We apply histogram modulation, a process that analyzes the bit distribution of chosen weights and adjusts only a limited number of bits to minimize alterations. In the decoding phase, ELLMark extracts bits from weights by an inverted encoding process. Then, it corrects error bits and watermark pieces by a twice-majority voting mechanism. Notably, all phases of our method are training-free with low hardware resources, making it particularly efficient for online platform implementations. We evaluated our technique on the LLaMA-3, OPT, and Phi-3 LLM families, demonstrating its adaptability. Our approach consistently achieves 100% success in detection while preserving model performance on various setups. Moreover, the preprocessing, encoding, and decoding processes are efficient, taking less than 7 minutes, 12 minutes, and 18 seconds, respectively, even for models with 80B parameters. It also shows robustness against various attacks, including parameter overwriting, re-watermarking, forging, fine-tuning, and pruning. Moreover, it has good watermark capacity and integrity. Lastly, compared to the state-of-the-art, the ELLMark supports full-precision and quantized models, as well as fine-tuning models, covering a broad range of models.

Contributions. (i) We propose an efficient white-box LLM watermarking for IP protection on an online market platform. (ii) We employ histogram modulation to encode watermarks and twice-majority voting to correct errors. (iii) Comprehensive experiments are performed to validate the effectiveness of our approach. The code is at repository¹.

The structure of the paper is as follows: Section 2 reviews related literature. Section 3 and Section 4 describe the problem statement and the designed approach. Section 5 presents the experimental results. Finally, Section 6 concludes the paper.

2 Related Work

Traditional Model Watermarking. The target of many traditional model watermarking techniques is DNN, which embed specific watermarks during training [41] [19] [42] [28] or as backdoors [35] [16] [50] to generate unintended outputs via trojan activations. However, they are hard to extend their capability to address LLMs with different architectures and billion parameters.

Large Model Watermarking. Zhao *et al.* [49] proposed a comprehensive analysis and practical guidelines for effectively watermarking DMs, either through training from scratch or fine-tuning. Li *et al.* [23] proposed the robust watermarking framework PLMmark

¹<https://github.com/Suxyuuu/LLM-WaterMark>

for PLMs, which establishes a link between the signature and triggers words and embeds task-agnostic watermarks via supervised contrastive loss. Peng *et al.* [34] proposed EmbMarker that embeds backdoors into embeddings for Embedding as a Service (EaaS) models. For LLMs, qiu *et al.* [22] co-optimize the full-precision and quantized LLMs to ensure watermarks as backdoors are in the full-precision model. Zhang and Koushanfar [47] proposed a robust watermarking algorithm, EmMark, which is designed to protect the IP of quantized LLMs on resource-constrained edge devices. EmMark ensures robustness and quality by strategically selecting watermark weight parameters. Li *et al.* [24] proposed a Double-I Watermark by leveraging backdoor samples into the dataset during LoRA fine-tuning. Oh *et al.* [32] propose the universal whitebox watermarking SEAL for LoRA fine-tuning that embeds a secret, non-trainable matrix between trainable LoRA weights.

Besides, there are many watermarking that protect LLM-generated text via logits generation [13, 20, 37] or token sampling [8] mechanisms, which are visible to adversaries. Thus, this type of techniques [8, 13, 20, 37] can not protect LLMs' IP. As demonstrated above, existing LLM watermarking approaches [22, 24, 32, 47] are largely limited to specific scenarios. Besides, these approaches typically require training processes to embed watermarks. The limited adaptability and high resource requirements create significant barriers to their effective implementation in online market platforms where efficiency and versatility are crucial.

3 Problem Statement and Threat Model

3.1 Problem Statement

As demand for IP protection in the online market, the watermarking technique is essential to prevent duplication, unauthorized resale, and commercial exploitation of the models. The watermarking system is illustrated in Figure 1, which consists of key components like the watermark system, user, and adversary. The online market can provide a watermark tool for users. When theft occurs, watermarks can provide copyright proof through this system. In online market platforms, an ideal watermarking framework should meet the criteria: (i) Adaptability, the watermark should be successfully embedded into diverse models. (ii) Fidelity, the watermarks should preserve model quality; (iii) Efficiency, the watermark process should be efficient both in terms of time and overheads; (iv) **Robustness, the watermarks should be robust against various attacks.**

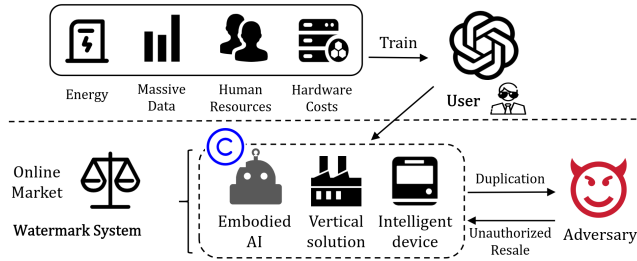


Figure 1: Watermark System in online market platform.

Table 1: Notions

Notion	Explanation	Notion	Explanation
M	LLM	M_w	Watermark LLM
WP	Watermark piece	W	Weight matrix of LLM
w	Weight in matrix W	R	Row in matrix W
SIG	Signature String	WM	Watermarks
CM	Candidate Matrix Set	L	Length of watermark
pn	Number of watermark pieces	pl	Length of watermark piece
X	Input feature activations	D	Calibration dataset
MB	Modifiable bit	PB	Position bit
γ	Density parameter	$Ratio$	Filter Ratio
$\mathcal{K}$	Secret key	δ, c	Watermark embedding threshold
$Hash$	Hash function	$\mathcal{S}$	Pseudo-random sequence generator

3.2 Threat Model

We assume the adversary has full access to the watermarked LLM's weights and is aware of the watermark encoding algorithms. However, the adversary cannot access the original LLM, owner's signature, and watermark parameters. Potential threats to models include: (i) parameter overwriting attacks, where model parameters are replaced with arbitrary values; (ii) re-watermarking attacks, where new watermarks are embedded by the adversary to falsely claim ownership; (iii) forging attacks, where counterfeit watermarks are created to misrepresent ownership; (iv) pruning attacks, where redundant or non-essential components are removed from the model; and (v) fine-tuning attacks, where the model is retrained on specific tasks or datasets, modifying its parameters accordingly.

4 Approach

4.1 Overview

The workflow of the ELLMark is illustrated in Figure 2. The watermark preprocessing assesses the significance of weights and then generates a candidate set of weights by filtering in phase 1. This filter is based on activation magnitude, input data, and key-based random selection. The white blocks with solid lines and with dashed lines are original and filtered weights, respectively. The watermark encoding process involves embedding the signature into the linear layers of LLM. The signature, containing the watermark's ownership information, is converted into a bit sequence. Then, the approach determines specific weight parameters and utilizes histogram modulation to embed the bits, creating the watermarked LLM in phase 2. The yellow blocks are candidate weights, while green blocks, including modified and unmodified weights, are watermarked weights. The watermark decoding is in phase 3, which is the inverted process of encoding. The grey blocks are extracted bits. The watermark decoding process entails detecting the embedded bits and comparing them to get a watermark piece. Finally, watermark bits 1010 are obtained by comparing the number of bits 1 and bit 0. The frequently utilized notations are presented in Table 1.

4.2 Preprocessing Process

The watermark preprocessing phase evaluates the significance of each weight value and develops a key-based random filter. The importance score for each weight is determined based on two criteria. The first criterion assesses the importance of weights based on their activation magnitudes. The saliency of a weight parameter is

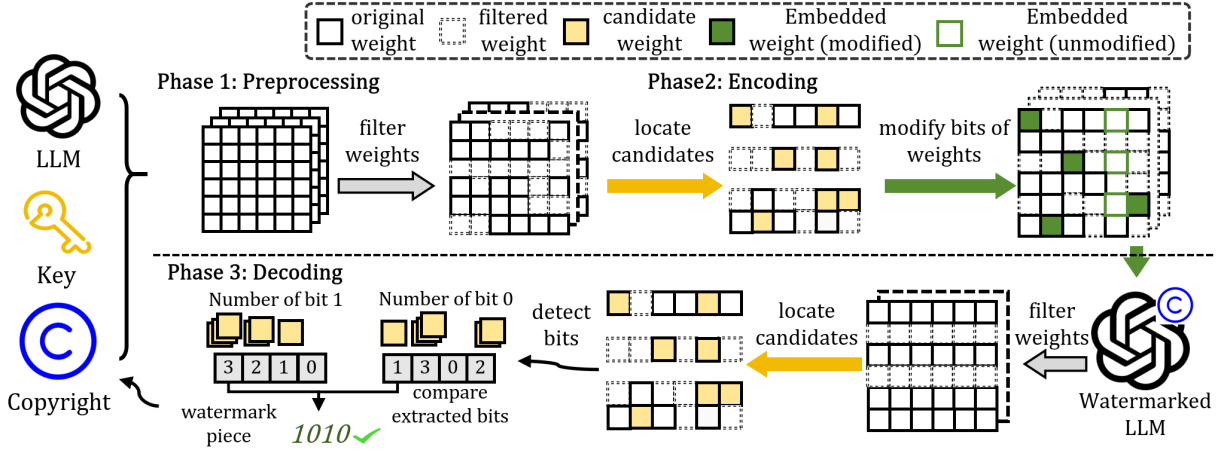


Figure 2: Workflow of ELLMark Approach.

closely linked to its activation magnitude; larger activations suggest a greater contribution of incoming features, thereby making the corresponding weight more significant [25]. The second criterion evaluates the correlations with downstream tasks using calibration data, ensuring that the watermarking process has a minimal impact on the performance of the main tasks. The score for the current weight $w_{i,j}$ is $S_{i,j}^1 = \frac{1}{|W_{i,j}| \cdot \|X_j\|^2}$, where $|W_{i,j}|$ represents the absolute value of $W_{i,j}$, $\|X_j\|^2$ evaluates the L_2 norm of j -th features aggregated across different tokens. The final score is computed as the inverse of the product of these two scalar values, facilitating the comparison and filtering of weights based on both criteria. This metric is valuable for several reasons. First, when the input channel of a given weight exhibits large magnitude activations, the weight tends to receive a higher importance score, even if its magnitude is low. This suggests that weights linked to significant features (those with strong correlations to downstream tasks) are more likely to be preserved. Second, the calculation of this metric is both straightforward and efficient; once the input feature activations are obtained, determining the weight's importance is a simple task.

Then we utilize a key-based random filter, which serves two primary objectives: (i) it enhances the watermark security by concealing the identity of the watermarked weights, making them less susceptible to unauthorized detection and removal attempts, and (2) it minimizes the number of weights targeted for watermarking, thereby preserving the overall quality of the LLM. It is important to note that $S_{i,j}^2$ is determined by the key $\mathcal{K}$ and can take values of either 1 or 0. The final score function S for each weight $w_{i,j}$ is

$$S_{i,j} = S_{i,j}^1 \cdot S_{i,j}^2 = \begin{cases} \frac{1}{|W_{i,j}| \cdot \|X_j\|^2}, & S_{i,j}^2 = 1; \\ 0, & S_{i,j}^2 = 0. \end{cases} \quad (1)$$

The preprocessing algorithm is detailed in Algorithm 1. For the weight w , a bigger score is better for watermark embedding. It starts by performing LLM inference, using input data from calibration dataset D . The calibration dataset should be representative of the model's typical tasks. If the calibration data is suitably selected, it does lead to better weight assessments. The input data is then

Algorithm 1 Preprocessing Algorithm

Input: LLM M , Dataset D , Key $\mathcal{K}$, Parameters γ , $Ratio$

Output: Candidate Set CM

- 1: **for** each matrix $W_l \in M$.decoders during LLM inference **do**
- 2: **if** $Hash(\mathcal{K}|l) \bmod \gamma_1 == 0$ **then**
- 3: $S^1 \leftarrow \frac{1}{|W_l| \cdot \|X\|^2}$ //get X from input feature activations
- 4: **for** each row $R_i \in W_l$ **do**
- 5: **if** $Hash(\mathcal{K}|i) \bmod \gamma_2 == 0$ **then**
- 6: For all $w_{i,j} \in R_i$, $S_{i,j}^2 \leftarrow 1$ and $S_{i,j} \leftarrow S_{i,j}^1 \cdot S_{i,j}^2$
- 7: **return** CM contains indexes of top $Ratio\%$ S

fed forward through a series of hidden layers. Each layer performs computations like linear transformation and activation functions. In Steps 2 and 5, the hash value $Hash(\mathcal{K}|l)$ and $Hash(\mathcal{K}|i)$ decide the selected matrix and its row, where $Hash$ is a cryptographic hash function [38] and $|$ is the concatenation operator. The γ_1 and γ_2 are density parameters for filtering matrices and their rows. After computation, X is the input feature activation for this weight matrix W . It intercepts the behavior of the LLM during its execution of fed forward at particular W . Finally, it evaluates weights' importance (Step 6) and selects the top $Ratio\%$ weights (Step 7).

4.3 Encoding Process

Ownership information SIG is used to generate an L -bits binary string $WM = \{b_0 b_1 b_2 \dots b_{L-1}\}$. Then, WM is divided into watermark pieces. Each piece WP has a part of signature bits. For instance, consider each piece consisting of an 8-bit binary string, where $WP_1 = \{b_0, b_1, \dots, b_7\}$ and $WP_2 = \{b_8, b_9, \dots, b_{15}\}$. The pl represents the length of each watermark piece, and pn denotes the total number of the watermark pieces.

4.3.1 Locating Method. Watermark locating methods are essential for identifying the specific watermark weight. Model weights come in various precision formats, ranging from FP32 to INT4. We define position bit (PB) and modifiable bits (MB) in Definition 1. Our algorithm determines the watermark based on the PB and embeds

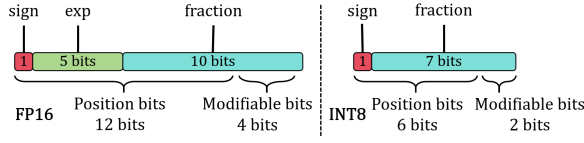


Figure 3: An example of position and modifiable bits.

it within MB of the host weight. While this approach is easy to implement and has a minimal impact on weight, it also exhibits robustness against attacks and is difficult to detect. For simplicity, we use FP16 and INT8 as examples to explain PB and MB². As shown in Figure 3, FP16, a half-precision floating point, uses 16 bits to represent a floating point number consisting of a 1-bit sign, a 5-bit exponent, and a 10-bit mantissa (fraction). For FP16 weight, we designate the leftmost 12 bits as PB and the rightmost 4 bits as MB. Besides, the PB and MB of INT8 are 6 and 2 bits in this case. The process for locating a weight is

$$S(\mathcal{K}|w.PB) \bmod \gamma = 0. \quad (2)$$

Pseudo-random number generator (PNG) S is a cryptographic tool for generating a sequence of numbers. The parameter γ controls the density of selected weights. Without the correct $\mathcal{K}$, it is impossible to recover ownership.

DEFINITION 1 (POSITION BITS AND MODIFIABLE BITS). Given a weight matrix, let w be the weight. The position bits refer to the bits that hold the larger values, typically represented by the leftmost PB bits. The modifiable bits refer to the bits that have the smaller values, typically the rightmost MB bits.

DEFINITION 2 (PSEUDO-RANDOM NUMBER GENERATOR). A PRNG S is an algorithm that produces a sequence of random numbers. A seed can determine the sequence. Let $S_i(\text{seed})$ denotes the i -th number in the sequence.

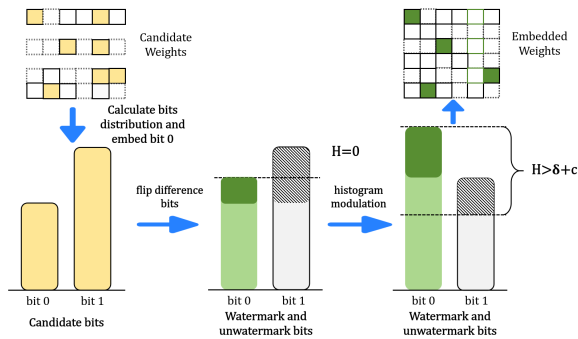


Figure 4: An example of histogram modulation.

²By default, FP16 is used for full precision, while INT8 is employed for quantized precision unless specified otherwise.

4.3.2 Histogram Modulation. Once the weights are located, MB is employed to embed the watermark information. We propose a histogram modulation technique that enables subtle modifications. It distorts an MB bit of the selected weight to encode the watermark while minimizing changes. In this scheme, all selected weights are divided into non-overlapping subsets, with each watermark bit embedded repetitively into one weight group. Within each group, HM is executed repetitively and is governed by two parameters: δ and c . δ serves as the watermark threshold, represented as a constant integer, while c is a small random integer. The embedding a bit b into a group G requires achieve this condition:

$$b = \begin{cases} 1, & \text{ocur}(1, G) - \text{ocur}(0, G) \geq \delta + c; \\ 0, & \text{ocur}(0, G) - \text{ocur}(1, G) \geq \delta + c. \end{cases} \quad (3)$$

The function $\text{ocur}(\cdot, G)$ represents the occurrence count of a specific bit in the weight group G . After this process, the frequency of bit b is $\delta + c$ greater than another bit. Detecting differs from the embedding process. It involves distinguishing the difference in bit distribution and identifying the bit with the highest frequency of occurrence. This approach offers two significant advantages: (i) it minimizes bit flipping, which preserves the quality of the LLM by avoiding modifications to all selected weights, and (ii) it enhances robustness against attacks by improving the error tolerance.

Example. Figure 4 illustrates an example of embedding bit 0 by HM. In this scenario, when we embed bit 0 into a weight group W , it calculates distributions of selected MB bits, and certain bits are flipped to ensure that the occurrences of bit 0 exceed those of bit 1 by $\delta + c$. H represents the quantity difference of bits 0 and 1. Besides, when detecting this bit, e.g., bit 0, from a group G , the condition $\text{ocur}(0, G) - \text{ocur}(1, G) \geq 0$ will satisfy.

4.3.3 Encoding Algorithm. The encoding algorithm is detailed in Algorithm 2. It begins by converting the signature string into watermark pieces. A loop traverses the weights in the decoders of LLM. Steps 2 and 4 check whether the current matrix and row are part of the candidate set CM . If the current object is not in this set, the algorithm skips to the next iteration. In Step 3, variables are defined. Specifically, $piece$ is the index of the watermark piece, I is a set containing records of the modified bits, and H is a set that records the bit differences of the selected weights. When the condition $S_1(\mathcal{K}, w.PB) \bmod \gamma_3 == 0$ is met, the current weight is selected for marking. A bit from the i -th position is then selected for modification. If the j -th bit of $W.MB$ equals x , and the value of $H[i]$ adds one; otherwise, it reduces one. This references the difference in bit distribution. In Step 14, for each watermark bit in WP_{piece} , the algorithm executes bit flipping until all values in H exceed $\delta + c$. This method flips only a subset of bits in the weights, ensuring both detectability and the preservation of model quality. Finally, after watermark embedding, the original LLM M is transformed into the watermarked LLM M_w .

4.4 Decoding Process

The decoding process is designed to detect signatures in algorithm 3. The M'_w of inputs is the watermarked LLM, which may be distorted under types of attacks. The other inputs are candidate set CM , signature string SIG , secret key $\mathcal{K}$, and parameters. It begins by

Algorithm 2 Encoding Algorithm

Input: LLM M , Signature SIG , Candidate CM , Key $\mathcal{K}$, Parameters $\delta, c, pn, pl, \gamma$

Output: Watermarked LLM M_w

```

1:  $WM \leftarrow \Psi(SIG)$ ,  $WP \leftarrow \text{split } WM \text{ into } pn \text{ pieces}$ 
2: for each matrix  $W_l \in M.\text{decoders}$  and  $W \in CM$  do
3:    $piece \leftarrow l \bmod pn$ ;  $I, H \leftarrow \{\}$ 
4:   for each row  $R \in W_l$  and  $R \in CM[l]$  do
5:     for each weight  $w \in R$  do
6:       if  $S_1(\mathcal{K}|w.PB) \bmod \gamma_3 == 0$  then
7:          $\text{markIndex } i \leftarrow S_2(\mathcal{K}|w.PB) \bmod pl$ 
8:          $\text{bitIndex } j \leftarrow S_3(\mathcal{K}|w.PB) \bmod |MB|$ 
9:         if  $j\text{-th bit of } w.MB == WP_{piece}[i]$  then
10:           $H[i] \leftarrow H[i] + 1$ 
11:       else
12:          $H[i] \leftarrow H[i] - 1$ ,  $I[i].\text{add}(\text{set } w.MB[j] \text{ to } x)$ 
13:   for  $i = 0$  to  $pl$  do
14:     flip bits according to  $I[i]$ , until  $H[i] > \delta + c$ 
15: return  $M_w$ 

```

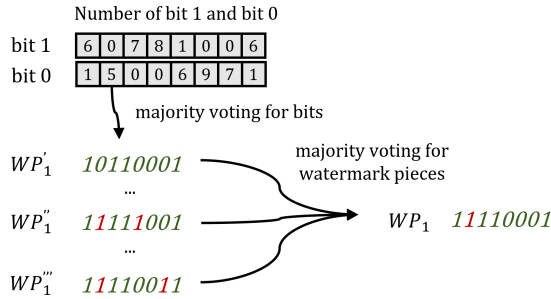


Figure 5: An example of twice-majority voting.

traversing the weight matrix and rows in steps 1-11. This step speeds up by skipping some of them. The two-dimensional array *ones, zeros* is initialed as the template of the pl -bits string. Then, it determines weight w , when $S_1(\mathcal{K}, w.PB) \bmod \gamma_3 == 0$. The watermark bits are extracted as the encoding algorithm does. Some error bits cannot generally decrease detection accuracy because each watermark bit will be embedded many times. By majority voting, bit errors are corrected, and the template of WP' is obtained in steps 12-13. Finally, it reconstructs watermark bit-string WM' by comparing watermark pieces WP' in step 14. Note that it uses twice-majority voting to correct errors. Figure 5 shows an example that determines bits from weights via first majority voting, then performs twice-majority voting across multiple watermark pieces.

4.5 Analysis

Security Analysis. The security of our watermarking system is achieved through encrypted algorithmic operations. While adversaries may attempt to remove watermarks by analyzing our algorithm through two main approaches: (i) analyzing the process of selecting the watermark locations and (ii) modifying the watermarked weights. Our system remains secure against both. For watermark location selection, we utilize a combination of hash

Algorithm 3 Decoding Algorithm

Input: Watermarked LLM M'_w , Signature SIG , Key $\mathcal{K}$, Parameters $\{\delta, pn, pl, \gamma\}$

Output: Detection Accuracy

```

1: for each  $W_l \in M'_w.\text{decoders}$  and  $\text{Hash}(\mathcal{K}|l) \bmod \gamma_1 == 0$  do
2:   for each  $R_i \in W_l$  and  $\text{Hash}(\mathcal{K}|i) \bmod \gamma_2 == 0$  do
3:      $\text{ones, zeros} \leftarrow []$ 
4:     for each weight  $w \in \text{Matrix do}$ 
5:       if  $S_1(\mathcal{K}|w.PB) \bmod \gamma_3 == 0$  and  $w! = \text{zeros}$  then
6:          $\text{markIndex } i \leftarrow S_2(\mathcal{K}|w.PB) \bmod pl$ 
7:          $\text{bitIndex } j \leftarrow S_3(\mathcal{K}|w.PB) \bmod |MB|$ 
8:         if  $j\text{-th bit of } w.MB == 1$  then
9:            $\text{ones}[i] ++$ 
10:        else
11:           $\text{zeros}[i] ++$ 
12:   for  $i = 0$  to  $pl-1$  do
13:      $WP'_l[i] \leftarrow \text{ones}[i] - \text{zeros}[i] >= 0 ? 1 : 0$ 
14:    $WM \leftarrow \Psi(SIG)$  and reconstruct  $WM'$  by  $WP'_1, \dots, WP'_{pn}$ 
15: return  $\frac{\text{match}(WM, WM')}{|WM|} * 100\%$ 

```

functions and PNG with a secret key, making it computationally infeasible to find different inputs producing identical outputs without possessing the secret key. Regarding weight modification attempts, adversaries face a significant challenge as they must maintain the LLM's performance while attempting to remove watermarks. Given that our filtering process results in a very small number of watermarked weights, it becomes impractical for adversaries to identify and modify these weights without compromising the performance.

Time and space complexity. Even our algorithm has several loops. In reality, the algorithm is efficient and has low complexity. Specifically, the time complexity of preprocessing and encoding is $O(|M|)$, where $|M|$ is the number of all weights. The complexity of decoding is $O(|M'|)$, where $O(|M'|)$ is the filtered M by excluding many matrices and rows. $O(|M'|)$ is about $\frac{|M|}{\gamma_1 \cdot \gamma_2}$. The memory and storage space is equivalent to the inference of LLM, and no additional requirement is required. The efficient and lightweight technique makes it scalable to LLMs with larger parameter sizes.

Watermark Density. On average, a fraction $\frac{|M| \cdot \text{Ratio\%}}{\gamma_1 \cdot \gamma_2 \cdot \gamma_3}$ of weights in model M will be selected for watermark embedding, where $|M|$ is the number of weights.

Blind and key-based feature. Blind means that the original LLM or watermark is not required for detection. Our algorithm ensures that the original LLM or watermark is not contained in the detection process. Hence, our algorithm is blind. Ownership information lies only in the choice of the secret key so that it is key-based.

Parameter influence. on average, weights will be selected to embed watermarks having possibility $\frac{\text{Ratio\%}}{\gamma_1 \cdot \gamma_2 \cdot \gamma_3}$. We say this value is ξ . We introduce parameter influence (API), which measures the mean change of a single weight. The API for FP16 and INT8 weight format are $\frac{2^{MB-1} \cdot \text{Ratio\%}}{2^e \cdot (m+1) \cdot \gamma_1 \cdot \gamma_2 \cdot \gamma_3}$ and $\frac{2^{MB-1} \cdot \text{Ratio\%}}{m \cdot \gamma_1 \cdot \gamma_2 \cdot \gamma_3}$. The MB is small. The parameter influence can be ignored. Details are in Appendix A.

False hit rate. It evaluates the probability that a non-watermarked model matches the signature by chance. There are Bernoulli trails.

Table 2: Effectiveness for Full-precision LLMs

Metrics	Methods	OPT					Llama-3		Phi-3			Δ
		1.3B	2.7B	6.7B	13B	30B	8B	70B	3.8B	7B	14B	
PPL ↓	Unwatermarked Model	14.63	12.47	10.85	10.12	9.55	6.14	2.85	6.35	7.30	4.30	0.00
	RandomMark	30785.29	13093.07	79100.30	43890.54	NaN	1272200.00	NaN	20476948.00	NaN	NaN	NaN
	RandomMark(10 bits)	14.63	12.47	10.85	10.12	9.55	6.14	2.85	6.35	7.30	4.30	0.00
	Quantize-Watermark	31.66	26.14	14.55	-	-	367.14	-	6.77	7.83	-	-66.06
	ELLMark	14.63	12.47	10.85	10.12	9.55	6.14	2.85	6.35	7.30	4.30	0.00
Zero-shot ACC (%) ↑	Unwatermarked Model	56.36	59.16	63.83	64.06	66.16	70.41	75.81	67.94	73.04	71.10	0.00
	RandomMark	31.03	31.03	31.03	31.03	31.03	31.03	31.03	31.03	31.03	31.03	-35.76
	RandomMark(10 bits)	56.30	59.15	63.82	64.06	66.13	70.41	75.80	67.86	73.02	71.02	-0.03
	Quantize-Watermark	51.0	55.02	59.34	-	-	69.43	-	64.81	71.02	-	-3.28
	ELLMark	56.29	59.15	63.80	64.07	66.14	70.41	75.81	67.89	73.04	71.07	-0.02
WER (%) ↑	ELLMark	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	-

Δ is the average performance degradation compared with non-watermarked models.

Table 3: Effectiveness for Quantized Llama-3 LLMs

Metrics	Methods	Llama-3		Δ
		8B	70B	
PPL ↓	Unwatermarked Model	5.57	2.84	-
	RandomMark	5.58	3.41	-0.29
	RandomMark(7 bits)	5.57	2.86	-0.01
	EmMark	5.64	2.86	-0.03
	ELLMark	5.57	2.85	-0.005
Zero-shot ACC (%) ↑	Unwatermarked Model	70.38	75.96	-
	RandomMark	70.29	68.20	-3.97
	RandomMark(7 bits)	70.36	75.96	-0.01
	EmMark	70.36	75.96	-0.01
	ELLMark	70.36	70.36	-0.01
WER (%) ↑	EmMark	100.00	100.00	-
	ELLMark	100.00	100.00	-

Δ is the average performance degradation compared with non-watermarked models.

It has only two probabilities, p and q , where $q = 1 - p$. Let $b(k; n, p)$ be the probabilities such that there are n trials, probabilities p for successful match one bit of watermark, and q for failure. k is the number of matching bits between the owner's and non-watermarked signatures. Because at least t watermark piece could construct watermark signature WM , the FHR of the signature is $FHR^{WM} = (\frac{pn}{2^{pl}} \prod_{i=0}^{pl-1} 2B(\lfloor \tau \omega_i \rfloor; \omega_i, p))^t$. The value of FHR^{WM} is close to 0 due to $pl \gg \log pn$. Details are in Appendix B.

5 Experiments

Model and Environment. The target LLMs are the LLaMA-3 [31], OPT [48], and Phi-3 [5] families, with parameter sizes ranging from 1.3B to 70B. The experiments were run on servers with a Ubuntu 22.04 system equipped with an NVIDIA A800 GPU (80GB RAM), an Intel Xeon E5-2680 CPU @ 2.80GHz, and 128GB of RAM.

Watermark Parameters. For target models, we embed T bits per matrix, where T is defined as $\frac{3h}{4}$. Here, h denotes the hidden size of a linear layer, ensuring both computational simplicity and fairness as compared with other approaches. For FP16 weights of target

models, the first 12 bits are assigned as PB, and the last 4 bits as MB. In the case of INT8 weights, the first 6 bits serve as PB, while the last 2 bits are designated as MB. The watermark signature consists of 32 bits, divided into 4 watermark pieces, each containing 8 bits. During preprocessing, we use the C4 dataset [36] as the calibration dataset. At the same time, the filter ratio is set at 75%.

Baselines. We choose EmMark [47] and Quantize-Watermark [22] as the scheme for comparison. EmMark is also a white-box watermarking scheme. We only compare EmMark on quantified models due to its limited application scenario. Quantize-Watermark [22] is on full-precision and small-size models. Besides, we implement two types of random modification, which simulate watermark embedding. RandomMark modifies all bits of selected weights, while RandomMark (10 bits) specifically alters only the rightmost 10 bits. **Metrics.** The performance of the watermarked LLM is assessed using *Perplexity (PPL)* on the WikiText dataset [30], and *Zero-shot Accuracy (Zero-shot Acc)* on LAMBADA [33], HellaSwag [45], PIQA [7], and WinoGrande datasets [15]. PPL assesses the model's predictive capability, with lower values indicating superior performance. Zero-shot Acc evaluates the model's ability to make accurate predictions or perform tasks without explicit training, where higher accuracy reflects better. The watermark extraction performance is evaluated using the percentage of signatures successfully extracted as *Watermark Extraction Rate (WER)*. The watermark efficiency is evaluated using the time and required memory. Notably, the details of experimental datasets are in Appendix C.

5.1 Effectiveness

Table 2 and 3 are the results for PPL, Zero-shot Accuracy, and WER in full-precision and quantized LLMs. For quantized LLMs, We use the Llama-3 quantized to INT8 by GPTQ [14] as the target.

Adaptability. Experimental LLMs are in different precision, i.e., FFP16 and INT8, and parameter sizes, i.e., from 1.3B to 80B, especially between full-precision and quantized models. LLMark maintains strong adaptability across these model families, achieving a WER of 100% across all setups.

Fidelity. ELLMark outperforms with no degradation in PPL and only 0.02% in Zero-shot Accuracy for full-precision LLMs. RandomMark approach leads to significant performance drops across all

Table 4: Algorithm Efficiency and Density Parameters

(a)											(b)						
Phases	OPT					Llama-3				Phi-3			Parameters	γ_1	γ_2	γ_3	Radio%
	1.3B	2.7B	6.7B	13B	30B	8B	70B	Q-8B	Q-70B	3.8B	7B	14B					
Preprocessing	38s	1m 1s	1m 33s	1m 55s	4m 14s	3m 46s	18m 50s	3m 52s	16m 36s	3m 15s	3m 48s	3m 27s					
Encoding	1m 7s	1m 47s	2m 6s	8m 2s	5m 40s	6m 26s	16m 35s	4m 9s	16m 33s	3m 7s	3m 2s	4m 59s					
Decoding	3s	3s	5s	18s	19s	10s	18s	11s	13s	18s	2s	3s					

m is the minutes, and s is the seconds. Q indicates the quantized model.

m is the minutes, and s is the seconds. Q indicates the quantized model.

models, with degradation rates of NaN and 35.76% for PPL and Zero-shot Accuracy, respectively. This indicates that not all weight bits are suitable for watermark embedding. At the same time, RandomMark (10 bits) shows a minimal degradation of 0.03% in Zero-shot Accuracy across most models. Quantize-Watermark is used on full-precision and small-size models (even for an 8B model, it requires 8*A100 and 20+ hours), which introduce the degradation. For quantized models, although EmMark [47] is specifically designed, ELLMark also achieves impressive results in this setting.

Efficiency. The table 4 (a) shows the efficiency of ELLMark across different models with varying sizes. The execution time reported does not include the loading time of the LLMs. For preprocessing, times range from 35s (smallest OPT model) to 18m 50s (70B Llama-3). Encoding takes longer, ranging from 1m 7s to 16m 35s. Decoding is notably faster, taking only 3-19 seconds across all models. Generally, larger models require more processing time, with the 70B Llama-3 model showing the longest processing times overall. All components of ELLMark are executed on GPUs, and the memory requirements are equivalent to running inference of LLM, with no additional storage or memory needed. Even for 80B sizes, it only takes a few minutes. This efficient watermarking technique ensures that ELLMark is scalable for diverse LLMs.

5.2 Impact of Parameters

Watermark Density. The embedding density is determined by parameters *Radio%* and $\gamma_1, \gamma_2, \gamma_3$, where the former determines the filtering rate, and the latter determines embedding density for blocks, layers and weights of LLM. As shown in Table 4 (b), we set a series of values for density parameters. ELLMark maintains a 100% WER. **Watermark Integrity.** With different parameters and secret keys, we test the LLMs from 1.3B to 80B to serve as both watermarked and non-watermarked LLMs. ELLMark successfully extracts all signatures from the watermarked models but fails to verify ownership of the non-watermarked ones.

Watermark Capacity. Table 5 illustrates the experiments conducted on the effect of signature length. We varied the length of the embedded bit from 0 to 200, increasing by 50 each time, with the experiments performed on Llama-3 8B models. From the table, we can see that the performance of the watermarked LLM remains consistent across various signature lengths with good performance.

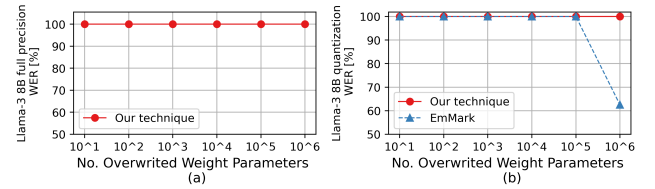
5.3 Potential Attacks

To strike a balance between experimental efficiency and representativeness, we utilize the Llama-3 8B model as our target model.

Table 5: Watermark Capacity

Llama-3 8B full precision model							Llama-3 8B quantization model						
Metrics	Signature Bit Length						Metrics	Signature Bit Length					
	0	50	100	150	200	Δ		0	50	100	150	200	Δ
PPL	6.14	6.14	6.14	6.14	6.14	0.00	PPL	5.57	5.57	5.57	5.57	5.57	0.00
Zero-shot ACC (%)	70.41	70.41	70.41	70.41	70.41	0.00	Zero-shot ACC (%)	70.36	70.36	70.36	70.36	70.36	0.00
WER (%)	-	100.00	100.00	100.00	100	0.00	WER (%)	-	100.00	100.00	100.00	100	0.00

Parameter Overwriting and Re-watermarking Attacks. The adversary may try to corrupt the watermarks by parameter overwriting, thereby hindering the detection process. In Figure 6, the number of overwritten parameters in each weight matrix increases from 10^1 to 10^6 . ELLMark exhibits robustness with 100% WER. Besides, the adversary attempts to confuse the original watermark on a watermarked LLM by applying a new watermark. These experiments are conducted on LLaMA-3 8B models in both full-precision and quantized formats. After multi-watermarking, ELLMark consistently maintains a high WER of 100%.

**Figure 6: Parameter overwriting attacks.**

Forging Attacks. The adversary does not attempt to remove the watermark of the LLM owner. Their methods include (i) counterfeiting the watermark weight locations with a fake signature sequence and (ii) re-watermarking the watermarked LLM. In the first scenario, the watermark weight locations are generated using the model's activations, calibration data, and a set of hyperparameters (including the secret key, filter ratio, and density parameter). Without these locating processes, the adversary fails to successfully launch forging attacks. In the second scenario, a key assumption for preventing ownership disputes is that both parties can extract their respective watermarks from each other's suspicious LLMs. However, this is not feasible, as the owner can effectively demonstrate the presence of their watermark in the adversary's LLM. In contrast, the adversary cannot provide evidence in the owner's LLM.

Fine-tuning Attacks. Fine-tuning involves adapting a pre-trained model to a specific task or dataset. Parameter-efficient fine-tuning (PEFT) techniques are frequently employed [17]. We selected several mainstream PEFT techniques. Prompt-tuning [21] optimizes

Table 6: FPFT and SFT Attacks

Method/Number	1k	10k	100k
FPFT	96.875	93.75	96.875
SFT	87.5	90.625	90.625

Table 7: Pruning Attacks

Methods	WER(%) ↑	Methods	WER(%) ↑
Wanda (10%)	100.00	Wanda (2:4)	90.63
Wanda (20%)	100.00	Wanda (4:8)	84.38
Wanda (30%)	100.00	LLM-Pruner (block-wise)	100.00
Wanda (40%)	100.00	LLM-Pruner (layer-wise)	100.00
Wanda (50%)	87.50		

For Wanda, pruning ratios are from 10% to 50% on global pruning, and it prunes A weights per B weights on group pruning (2:4 and 4:8). For LLM-Pruner, block-wise changes, the number of decoder blocks, and layer-wise changes the number of layers.

prompts to guide a pre-trained language model’s responses, tailoring its output for specific tasks without altering the parameters. P-tuning-v2 [26] extends prompt-tuning by using a small network (0.1% - 3%) to dynamically generate prompts based on the input text. BitFit [44] focuses on adjusting a small subset of the model’s parameters, specifically the biases within the layers. LoRA [18] incorporates low-rank adapters to learn features from new datasets. ELLMark maintains a 100% WER even after these four fine-tuning attacks, which indicates fine-tuning attacks was unsuccessful. Besides, we test full-parameter fine-tuning (FPFT) and Supervised Fine-Tuning (SFT) ³. In Table 6, our method achieves high WER.

Pruning Attacks. Pruning involves removing redundant or less important parameters from a model, leading to faster inference, lower memory usage, and often improved generalization. We selected several mainstream pruning methods. Wanda [40] evaluates and removes weights based on the product of their magnitude and the norm of the corresponding input activations. LLM-pruner [29] employs a dependency detection algorithm to identify interdependent structures within the model. As shown in Table 7, ELLMark achieves high WER after pruning attacks. Notably, fine-tuning and pruning cannot be applied to quantized LLMs, as these methods can disrupt model and lead to chaotic outputs. Consequently, related experiments are conducted on full-precision models.

5.4 Comparison with State-of-the-art

There are only a few watermarking techniques for LLMs’ IP protection. Table 8 shows the details across different scenarios. EmMark [47] is for quantized models, while the quantize-watermark [22] works when the full-precision model is after the quantization process. Quantize-watermark uses FPFT to inject watermark thus requiring high hardware resource. Besides, approaches [24, 32] are adaptable for only LoRA fine-tuning models with smaller parameter sizes like 2B and 7B. In this section, we have experimented with

³The experiments were conducted on 3×A100 GPUs using AdamW optimizer (learning rate=2e⁻⁵) for 1 epoch on both IMDB and chatbot_instruction_prompts datasets, respectively.

Table 8: Comparisons

Approach	Scenario	Llama-3 8B		
		Full precision model	Quantized model	LoRA fine-tuning model
EmMark [47]	Quantized models	○	●	○
Quantize-Watermark [22]	Full-precision model	●	○	○
Double-I [24]	LoRA fine-tuning model	○	○	●
SEAL [32]	LoRA fine-tuning model	○	○	●
ELLMark	Full-precision and quantized models including fine-tuning	●	●	●

●, ○ and ○ indicate “yes”, “partially yes” and “no” respectively.

full-precision and quantized models. Besides, we add an experiment on the LoRA fine-tuning model of Llama-3 8B, and ELLMark obtains 100% WER with degradation of 0 for PPL and Zero-shot Accuracy. Existing LLM watermarking approaches are largely limited to online market platforms. The ELLMark approach supports full-precision and quantized models, as well as fine-tuning, covering a broad range of models.

6 Discussion and Conclusion

About quantization model. Weight quantization involves converting the floating-point weights of the model into lower-precision representations. However, weight does not have sufficient bits for assigning position bits and modifiable bits. This creates two problems: (i) embedded weights can not be accurately located due to the limited length of the position bit; (ii) embedded weights may be easily distorted under attacks due to modifiable bits being fixed at the leftmost position. Thus, for INT4 and even more quantized weight, we can select more than one weight as a watermark weight to increase the weights’ bits for locating and embedding.

About quantization attack. When a watermarked model undergoes quantization (e.g., from 16-bit float to 8-bit integer), the watermark could be destroyed. For this attack, we selectively increase the range of modified bits, which ensures that the quantized weights also change. However, this method stores the deviation (i.e., positive or negative changes) as a reference.

Adaptability for LLMs. Our watermarking technique can extend to other LLMs because we embed signatures into the weights of linear layers. These linear layers are basic structures that are prevalent and numerous within LLMs. Besides, our techniques are plug-and-play, which does not depend on additional requirements.

Conclusion. We introduce a novel watermarking technique specifically designed to protect LLMs. Our proposed method embeds watermarks directly into the properly selected model weights without requiring any retraining while ensuring minimal performance degradation. Extensive testing on various models, including LLaMA-3, OPT, and Phi-3, has demonstrated its effectiveness.

7 Acknowledgements

This research is supported by the Strategic Priority Research Program of the Chinese Academy of Sciences, Grant No. XDB0690303.

References

- [1] [n. d.]. *Hugging Face*. <https://huggingface.co/> 2022.
- [2] [n. d.]. *Model Zoo*. <https://modelzoo.co/> 2021.
- [3] [n. d.]. *ONNX*. <https://onnx.ai/> 2022.

- [4] [n. d.]. *What are LLMs?* <https://www.ibm.com/topics/large-language-models> 2024.
- [5] Marah Abdin, Sam Ade Jacobs, Ammar Ahmad Awan, Jyoti Aneja, Ahmed Awadallah, Hany Awadallah, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Jianmin Bao, Harkirat Behl, Alon Benhaim, Misha Bilenko, Johan Bjorck, Sébastien Bubeck, Qin Cai, Martin Cai, Caio César Teodoro Mendes, Weizhu Chen, Vishrav Chaudhary, Dong Chen, Dongdong Chen, Yen-Chun Chen, Yi-Ling Chen, Parul Chopra, Xiyang Dai, Allie Del Giorno, Gustavo de Rosa, Matthew Dixon, Ronen Eldan, Victor Fragoso, Dan Iter, Mei Gao, Min Gao, Jianfeng Gao, Amit Garg, Abhishek Goswami, Suriya Gunasekar, Emman Haider, Junheng Hao, Russell J. Hewett, Jamie Huynh, Mojan Javaheripi, Xin Jin, Piero Kauffmann, Nikos Karampatziakis, Dongwoo Kim, Mahoud Khademi, Lev Kurilenko, James R. Lee, Yin Tat Lee, Yuanzhi Li, Yunsheng Li, Chen Liang, Lars Liden, Ce Liu, Mengchen Liu, Weishung Liu, Eric Lin, Zeqi Lin, Chong Luo, Piyush Madan, Matt Mazola, Arindam Mitra, Hardik Modi, Anh Nguyen, Brandon Norick, Barun Patra, Daniel Perez-Becker, Thomas Portet, Reid Pryzant, Heyang Qin, Marko Radmilac, Corby Rosset, Sambudha Roy, Olatunji Ruwase, Olli Saarikivi, Amin Saied, Adil Salim, Michael Santacrose, Shital Shah, Ning Shang, Hiteshi Sharma, Swadheer Shukla, Xia Song, Masahiro Tanaka, Andrea Tupini, Xin Wang, Lijuan Wang, Chunyu Wang, Yu Wang, Rachel Ward, Guanhua Wang, Philipp Witte, Haiping Wu, Michael Wyatt, Bin Xiao, Can Xu, Jiahang Xu, Weijian Xu, Sonali Yadav, Fan Yang, Jianwei Yang, Ziyi Yang, Yifan Yang, Donghan Yu, Lu Yuan, Chengruidong Zhang, Cyril Zhang, Jianwen Zhang, Li Lyna Zhang, Yi Zhang, Yue Zhang, Yunan Zhang, and Xiren Zhou. 2024. Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone. *arXiv:2404.14219* [cs.CL] <https://arxiv.org/abs/2404.14219>
- [6] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [7] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. 2020. PIQA: Reasoning about Physical Commonsense in Natural Language. In *Thirty-Fourth AAAI Conference on Artificial Intelligence*.
- [8] Miranda Christ, Sam Gunn, and Or Zamir. 2024. Undetectable watermarks for language models. In *The Thirty-Seventh Annual Conference on Learning Theory*. PMLR, 1125–1139.
- [9] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168* (2021).
- [10] Tianshuo Cong, Delong Ran, Zesen Liu, Xinlei He, Jinyuan Liu, Yichen Gong, Qi Li, Anyu Wang, and Xiaoyun Wang. 2023. Have you merged my model? on the robustness of large language model ip protection methods against model merging. In *Proceedings of the 1st ACM Workshop on Large AI Systems and Models with Privacy and Safety Analysis*. 69–76.
- [11] Bitu Darvish Rouhani, Huili Chen, and Farinaz Koushanfar. 2019. Deepsigns: An end-to-end watermarking framework for ownership protection of deep neural networks. In *Proceedings of the twenty-fourth international conference on architectural support for programming languages and operating systems*. 485–497.
- [12] Danny Driess, Fei Xia, Mehdi SM Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, et al. 2023. Palm-e: An embodied multimodal language model. *arXiv preprint arXiv:2303.03378* (2023).
- [13] Jaiden Fairoze, Sanjam Garg, Somesh Jha, Saeed Mahloujifar, Mohammad Mahmoody, and Mingyuan Wang. 2023. Publicly detectable watermarking for language models. *arXiv preprint arXiv:2310.18491* (2023).
- [14] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2023. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. *arXiv:2210.17323* [cs.LG] <https://arxiv.org/abs/2210.17323>
- [15] L Gao, J Tow, B Abbasi, S Biderman, S Black, A DiPofi, C Foster, L Golding, J Hsu, A Le Noac'h, et al. [n. d.]. A framework for few-shot language model evaluation, 12 2023. URL <https://zenodo.org/records/10256836> 7 ([n. d.]).
- [16] Jia Guo and Miodrag Potkonjak. 2018. Watermarking deep neural networks for embedded systems. In *2018 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 1–8.
- [17] Zeyu Han, Chao Gao, Jinyang Liu, Sai Qian Zhang, et al. 2024. Parameter-efficient fine-tuning for large models: A comprehensive survey. *arXiv preprint arXiv:2403.14608* (2024).
- [18] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. LoRA: Low-Rank Adaptation of Large Language Models. *arXiv:2106.09685*
- [19] Hengrui Jia, Christopher A Choquette-Choo, Varun Chandrasekaran, and Nicolas Papernot. 2021. Entangled watermarks as a defense against model extraction. In *30th USENIX security symposium (USENIX Security 21)*. 1937–1954.
- [20] John Kirchenbauer, Jonas Geiping, Yuxin Wen, Jonathan Katz, Ian Miers, and Tom Goldstein. 2023. A watermark for large language models. In *International Conference on Machine Learning*. PMLR, 17061–17084.
- [21] Brian Lester, Rami Al-Rfou, and Noah Constant. 2021. The power of scale for parameter-efficient prompt tuning. *arXiv preprint arXiv:2104.08691* (2021).
- [22] Linyang Li, Botian Jiang, Pengyu Wang, Ke Ren, Hang Yan, and Xipeng Qiu. 2023. Watermarking LLMs with Weight Quantization. *arXiv preprint arXiv:2310.11237* (2023).
- [23] Peixuan Li, Pengzhou Cheng, Fangqi Li, Wei Du, Haodong Zhao, and Gongshen Liu. 2023. Plmmark: a secure and robust black-box watermarking framework for pre-trained language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 14991–14999.
- [24] Shen Li, Liuyi Yao, Jinyang Gao, Lan Zhang, and Yaliang Li. 2024. Double-I Watermark: Protecting Model Copyright for LLM Fine-tuning. *arXiv:2402.14883* [cs.CR] <https://arxiv.org/abs/2402.14883>
- [25] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration. *Proceedings of Machine Learning and Systems 6* (2024), 87–100.
- [26] Xiao Liu, Kaixuan Ji, Yicheng Fu, Weng Lam Tam, Zhengxiao Du, Zhilin Yang, and Jie Tang. 2021. P-tuning v2: Prompt tuning can be comparable to fine-tuning universally across scales and tasks. *arXiv preprint arXiv:2110.07602* (2021).
- [27] Yixin Liu, Hongsheng Hu, Xuyun Zhang, and Lichao Sun. 2023. Watermarking text data on large language models for dataset copyright protection. *arXiv preprint arXiv:2305.13257* (2023).
- [28] Peizhuo Lv, Pan Li, Shengzhi Zhang, Kai Chen, Ruigang Liang, Hualong Ma, Yue Zhao, and Yingjiu Li. 2023. A robustness-assured white-box watermark in neural networks. *IEEE Transactions on Dependable and Secure Computing* (2023).
- [29] Xinyin Ma, Gongfan Fang, and Xinchao Wang. 2023. Llm-pruner: On the structural pruning of large language models. *Advances in neural information processing systems 36* (2023), 21702–21720.
- [30] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2016. Pointer Sentinel Mixture Models. *arXiv:1609.07843* [cs.CL] <https://arxiv.org/abs/1609.07843>
- [31] Meta. [n. d.]. *Llama 3*. <https://llama.meta.com/llama3/> 2023.
- [32] Giyeong Oh, Saejin Kim, Woohyun Cho, Sangkyu Lee, Jiwan Chung, Dokyung Song, and Youngjae Yu. 2025. SEAL: Entangled White-box Watermarks on Low-Rank Adaptation. *arXiv:2501.09284* [cs.AI] <https://arxiv.org/abs/2501.09284>
- [33] Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Ngoc Quan Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernandez. 2016. The LAMBADA dataset: Word prediction requiring a broad discourse context. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Berlin, Germany, 1525–1534. <http://www.aclweb.org/anthology/P16-1144>
- [34] Wenjun Peng, Jingwei Yi, Fangzhao Wu, Shangxi Wu, Bin Zhu, Lingjuan Lyu, Binxing Jiao, Tong Xu, Guangzhong Sun, and Xing Xie. 2023. Are you copying my model? protecting the copyright of large language models for eas via backdoor watermark. *arXiv preprint arXiv:2305.10036* (2023).
- [35] Yuhui Quan, Huan Teng, Yixin Chen, and Hui Ji. 2020. Watermarking deep neural networks in image processing. *IEEE transactions on neural networks and learning systems 32*, 5 (2020), 1852–1865.
- [36] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *J. Mach. Learn. Res.* 21, 1, Article 140 (jan 2020), 67 pages.
- [37] Jie Ren, Han Xu, Yiding Liu, Yingqian Cui, Shuaiqiang Wang, Dawei Yin, and Jiliang Tang. 2023. A robust semantics-based watermark for large language model against paraphrasing. *arXiv preprint arXiv:2311.08721* (2023).
- [38] Bruce Schneier. 2007. *Applied cryptography: protocols, algorithms, and source code in C*. John Wiley & sons.
- [39] Alexandra Souly, Qingyuan Lu, Dillon Bowen, Tu Trinh, Elvis Hsieh, Sana Pandey, Pieter Abbeel, Justin Svegliato, Scott Emmons, Olivia Watkins, and Sam Toyer. 2024. A StrongREJECT for Empty Jailbreaks. *arXiv:2402.10260* [cs.LG] <https://arxiv.org/abs/2402.10260>
- [40] Mingjie Sun, Zhuang Liu, Anna Bair, and J. Zico Kolter. 2023. A Simple and Effective Pruning Approach for Large Language Models. *CoRR* abs/2306.11695 (2023). doi:10.48550/ARXIV.2306.11695
- [41] Yusuke Uchida, Yuki Nagai, Shigeyuki Sakazawa, and Shin'ichi Satoh. 2017. Embedding watermarks into deep neural networks. In *Proceedings of the 2017 ACM on international conference on multimedia retrieval*. 269–277.
- [42] Tianhao Wang and Florian Kerschbaum. 2021. Riga: Covert and robust white-box watermarking of deep neural networks. In *Proceedings of the Web Conference 2021*. 993–1004.
- [43] Chenfei Wu, Shengming Yin, Weizhen Qi, Xiaodong Wang, Zecheng Tang, and Nan Duan. 2023. Visual chatgpt: Talking, drawing and editing with visual foundation models. *arXiv preprint arXiv:2303.04671* (2023).
- [44] Elad Ben Zaken, Shauli Ravfogel, and Yoav Goldberg. 2021. BitFit: Simple Parameter-efficient Fine-tuning for Transformer-based Masked Language-models. *arXiv preprint arXiv:2106.10199* (2021).
- [45] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. HellaSwag: Can a Machine Really Finish Your Sentence?. In *Proceedings of the*

- 57th Annual Meeting of the Association for Computational Linguistics.
- [46] Junjie Zhang, Ruobing Xie, Yupeng Hou, Wayne Xin Zhao, Leyu Lin, and Ji-Rong Wen. 2023. Recommendation as instruction following: A large language model empowered recommendation approach. *arXiv preprint arXiv:2305.07001* (2023).
- [47] Ruisi Zhang and Farinaz Koushanfar. 2024. EmMark: Robust Watermarks for IP Protection of Embedded Quantized Large Language Models. arXiv:2402.17938 [cs.CR]
- [48] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, Todor Mihaylov, Myale Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Singh Koura, Anjali Sridhar, Tianlu Wang, and Luke Zettlemoyer. 2022. OPT: Open Pre-trained Transformer Language Models. arXiv:2205.01068 [cs.CL] <https://arxiv.org/abs/2205.01068>
- [49] Yunqing Zhao, Tianyu Pang, Chao Du, Xiao Yang, Ngai-Man Cheung, and Min Lin. 2023. A recipe for watermarking diffusion models. *arXiv preprint arXiv:2303.10137* (2023).
- [50] Renjie Zhu, Xinpeng Zhang, Mengte Shi, and Zhenjun Tang. 2020. Secure neural network watermarking protocol against forging attack. *EURASIP Journal on Image and Video Processing* 2020 (2020), 1–12.

A Parameter Influence

For each embedded weight, it modifies fraction value, having maximum value $2^{MB} - 1$. FP16 has a 1-bit sign (*sign*), a 5-bit exponent, and a 10-bit mantissa. The influence of an FP16 weight is at most $2^{MB} - 1$. Namely, this weight's 5-bit exponent is 00000, and the 10-bit mantissa is 00....0. The minimum influence is 0, not modifying weight. The upper bound is

$$PI_{upper} = (2^{MB} - 1) \cdot \xi \\ = \frac{(2^{MB} - 1) \cdot \text{Ratio}\%}{\gamma_1 \cdot \gamma_2 \cdot \gamma_3}. \quad (4)$$

Assume it is in FP16 weight format. The average value of exponent and mantissa are e and m . The average parameter influence (API) for the FP16 weight format is

$$API_{FP16} = \frac{2^{MB-1}}{2^e \cdot (m+1)} \cdot \xi \\ = \frac{2^{MB-1} \cdot \text{Ratio}\%}{2^e \cdot (m+1) \cdot \gamma_1 \cdot \gamma_2 \cdot \gamma_3}. \quad (5)$$

The average parameter influence (API) for INT8 weight format is

$$API_{INT8} = \frac{2^{MB-1}}{m} \cdot \xi \\ = \frac{2^{MB-1} \cdot \text{Ratio}\%}{m \cdot \gamma_1 \cdot \gamma_2 \cdot \gamma_3}. \quad (6)$$

The MB is small. Hence, the parameter influence can be ignored.

B False hit rate

It evaluates the probability that a non-watermarked model matches the watermark signature by chance. It has only two probabilities, p and q , where $q = 1 - p$. Let $b(k; n, p)$ be the probabilities such that there are n trials, probabilities p for successful match one bit of watermark, and q for failure. k is the number of matching bits between the owner's and non-watermarked model's signatures. Repeated independent trials are Bernoulli trials. Then

$$b(k; n, p) = \binom{n}{k} p^k q^{n-k} \quad (7)$$

The cumulative binomial probability is

$$B(k; n, p) = \sum_{i=k+1}^n b(i; n, p) \quad (8)$$

Thus, the probability of a possible watermark piece is $\prod_{i=0}^{pl-1} 2B(\lfloor \tau \omega_i \rfloor; \omega_i, p)$, where the factor 2 is that each bit may be either zero or one, ω_i is extracted times of b_i , τ is the detection threshold. The number of possible binary strings is 2^{pl} , and pl is the length of the watermark piece. The number of watermark pieces is pn . Thus, a binary string matches a valid watermark piece is $\frac{pn}{2^{pl}}$. For a watermark piece WP , the overall false hit rate is

$$FHR^{WP} = \frac{pn}{2^{pl}} \prod_{i=0}^{pl-1} 2B(\lfloor \tau \omega_i \rfloor; \omega_i, p) \\ = pn \prod_{i=0}^{pl-1} \left(\sum_{j=\lfloor 0.5\omega_i \rfloor+1}^{\omega_i} b(\lfloor 0.5\omega_i \rfloor; \omega_i, 0.5) \right) \quad (9)$$

Because at least t watermark piece could construct watermark signature WM , the overall false hit rate of the signature is

$$FHR^{WM} = \left(\frac{pn}{2^{pl}} \prod_{i=0}^{pl-1} 2B(\lfloor \tau \omega_i \rfloor; \omega_i, p) \right)^t \\ = \left(pn \prod_{i=0}^{pl-1} \left(\sum_{j=\lfloor 0.5\omega_i \rfloor+1}^{\omega_i} b(\lfloor 0.5\omega_i \rfloor; \omega_i, 0.5) \right) \right)^t \quad (10)$$

The false hit rate can be reduced exponentially by increasing pl . The value of FHR^{WM} is close to 0 if $pl \gg \log pn$. Hence, the false hit rate of our algorithm is very low.

C Experiment Dataset

We describe more information of datasets on Section 4. Specifically, we now provide background information on each dataset's tasks and relevance to evaluating model performance in the context of watermarking. This additional information should provide a clearer understanding of the datasets' roles in our evaluation and the significance of the results.

Our algorithm utilizes C4 dataset as a calibration dataset to assist filtering of weights.

- **C4.** A data set consisting of hundreds of gigabytes of clean English text scraped from the web.

Perplexity evaluates text fluency on the following datasets.

- **WikiText.** The full WikiText dataset is over 103 million words in size, a hundred times larger than the PTB. It is also one of the largest publicly available language modeling benchmarks, consisting of articles that allow for the capture and usage of longer-term dependencies, which might be found in many real-world tasks.

Zero-shot Acc to measure token prediction, averaged across the LAMBADA, HellaSwag, PIQA, and WinoGrande datasets.

- **LAMBADA.** A data set is a collection of narrative passages. LAMBADA proposes a word prediction task where the target item is difficult to guess (for English speakers) when only the sentence in which it appears is available but becomes easy when a broader context is presented. LAMBADA

casts language understanding in the classic word prediction framework of language modeling.

- **HellaSwag.** A data set is used to perform a word prediction task. Though its questions are trivial for humans (95% accuracy), state-of-the-art models struggle (48%).
- **PIQA.** A data set is used to evaluate language representations based on their knowledge of physical commonsense. Though humans find the dataset easy (95% accuracy), large pre-trained models struggle (77%). The underlying task is multiple choice question answering: given a question q and two possible solutions, $s1$ and $s2$, a model or a human must choose the most appropriate solution, of which exactly one is correct.
- **WinoGrande.** A data set is a large-scale, high-quality benchmark designed for evaluating commonsense reasoning and pronoun resolution in language models. The dataset is particularly used for testing the model's ability to understand and resolve ambiguous pronouns in natural language sentences.

StrongReject evaluates the safety alignment; GSM8K evaluates the LLMs' ability in Mathematics.

- **StrongReject.** StrongReject-small is a highquality dataset that contains 50 forbidden questions which cover 6 widely prohibited types of misuse, including (1) Illegal goods and services, (2) Hate/harassment/discrimination, (3) non-violent

crimes, (4) Disinformation and Deception, (5)Violence, and (6) Sexual content.

- **GSM8K.** GSM8K consists of high quality grade school level manually crafted mathematics problems.

D Supplementary Experiments

We conduct experiments about the functionality of watermarked models across different applications on LLama3 Instruct-8B. StrongReject [39] evaluates the safety alignment; GSM8K [9] evaluates the LLMs' ability in Mathematics; LAMBADA, HellaSwag, PIQA, and WinoGrande evaluate the common sense. StrongReject uses metric substring lookup [10] to judge if LLMs successfully refuse to answer the forbidden questions; GSM8K uses answer accuracy [10] to evaluate. Other datasets use zero-shot accuracy. The table 9 shows the result and change, which has tiny even 0 degradation for these tasks.

Table 9: Functionality of Watermarked Models

Model/Dataset	StrongReject	GSM8K	LAMBADA	HellaSwag	PIQA	Wino
Watermarked Model	0.56	0.53	68.79	60.16	79.71	72.77
Degradation	-0.00	-0.01	-0.00	-0.00	-0.00	-0.08